

Speedy-Splat: Fast 3D Gaussian Splatting with Sparse Pixels and Sparse Primitives

Alex Hanson

Allen Tu

Geng Lin

Vasu Singla

Matthias Zwicker

Tom Goldstein

University of Maryland, College Park

<https://speedysplat.github.io>

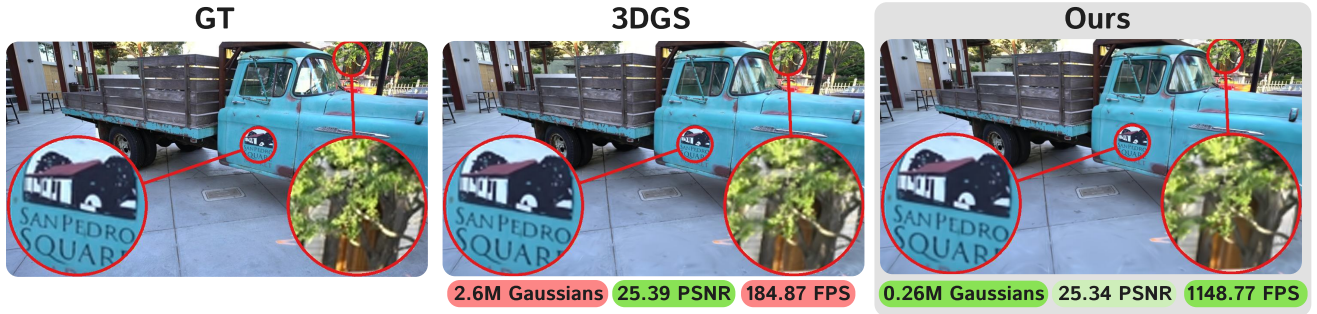


Figure 1. We reduce the number of Gaussians by over 90%, only marginally decrease PSNR, and accelerate rendering speed by $6.2\times$ in the Tanks & Temples truck scene when compared to 3D Gaussian Splatting (3D-GS). Additionally, we speed up training time by $1.38\times$.

Abstract

3D Gaussian Splatting (3D-GS) is a recent 3D scene reconstruction technique that enables real-time rendering of novel views by modeling scenes as parametric point clouds of differentiable 3D Gaussians. However, its rendering speed and model size still present bottlenecks, especially in resource-constrained settings. In this paper, we identify and address two key inefficiencies in 3D-GS to substantially improve rendering speed. These improvements also yield the ancillary benefits of reduced model size and training time. First, we optimize the rendering pipeline to precisely localize Gaussians in the scene, boosting rendering speed without altering visual fidelity. Second, we introduce a novel pruning technique and integrate it into the training pipeline, significantly reducing model size and training time while further raising rendering speed. Our Speedy-Splat approach combines these techniques to accelerate average rendering speed by a drastic $6.71\times$ across scenes from the Mip-NeRF 360, Tanks & Temples, and Deep Blending datasets.

1. Introduction

Fast rendering of photorealistic novel views has been a long-standing goal in computer vision and graphics. Neural

Radiance Fields (NeRF) [23] and its variants have made significant strides in photorealistic 3D scene reconstruction by representing scenes as continuous neural volumetric models that encode scene density and color at spatial coordinates. However, despite recent efforts [4, 24, 29], fast rendering in NeRF remains challenging because the volumetric sampling used in ray-marching is computationally expensive. Recently, 3D Gaussian Splatting (3D-GS) [14] has emerged as a promising alternative that enables real-time rendering by modeling scenes as parametric point clouds of differentiable 3D Gaussians. Nevertheless, its rendering speed is still limited by high parameter counts and certain algorithmic inefficiencies. Efficient rendering is essential in applications such as virtual reality, networked systems, and multi-view streaming. Real-time rendering on resource-constrained edge devices, such as mobile phones, has yet to be achieved [19].

Although recent works on compressing 3D-GS models [6, 7, 11, 25] achieve some speed-ups by reducing the number of parameters, few approaches directly target rendering speed [8, 19]. In this paper, we specifically address this gap by demonstrating that the rendering speed of 3D-GS models can be drastically increased while maintaining competitive image quality. Additionally, we show that our methods reduce training time and substantially decrease model size.

We begin by observing that the cost of 3D-GS rendering

is proportional to both the number of Gaussians in the scene and the number of pixels processed per Gaussian. Our approach optimizes both factors. First, we reduce the number of pixels that are processed for each Gaussian by efficiently and accurately localizing it in the rendered image. Second, we reduce the total number of Gaussians in the model through a novel approach that maintains rendering quality.

3D-GS implements tile rendering to localize Gaussians in the image plane by assigning each Gaussian to the tiles that it intersects. We find the existing algorithm to be overly conservative; in Section 4.1.1, we address this by introducing our SnugBox algorithm to precisely localize Gaussians by computing a tight bounding box around their extent. Then, in Section 4.1.2, we extend SnugBox with our AccuTile algorithm to identify exact tile intersections. Both approaches are plug-and-play, lead to respective inference speed-ups of $1.82\times$ and $1.99\times$ on average, and do not change the 3D-GS renderings.

To reduce the total number of Gaussians while preserving visual fidelity, we extend an existing pruning method [11] by reducing its memory requirement by $36\times$ and incorporating it into the 3D-GS training pipeline. During the densification stage of 3D-GS training, Gaussians are regularly replicated and pruned. In Section 4.2.2, we augment the densification stage with our Soft Pruning method to prune 80% of Gaussians at regular intervals. After the densification stage, our Hard Pruning method, described in Section 4.2.3, prunes an additional 30% of Gaussians at set intervals.

Our Speedy-Splat approach integrates both techniques into the 3D-GS training pipeline. On average, rendering speed is accelerated by $6.71\times$, model size is reduced by $10.6\times$, and training speed is improved by $1.4\times$ across all evaluated scenes while maintaining high image quality.

In summary, we propose the following contributions:

1. **SnugBox**: A precise algorithm for computing Gaussian-tile bounding box intersections.
2. **AccuTile**: An extension of SnugBox for computing exact Gaussian-tile intersections.
3. **Soft Pruning**: An augmentation for pruning Gaussians during densification.
4. **Hard Pruning**: An augmentation for pruning Gaussians post-densification.

2. Related work

The real-time rendering speed of 3D-GS [14] on desktop GPUs has inspired research focused on further accelerating both its training and inference in resource-constrained environments. In this section, we review related works that specifically target these performance improvements.

2.1. Pruning

A large portion of Gaussians in vanilla 3D-GS models are redundant [6, 7, 25], motivating a recent line of work focused on pruning Gaussians from 3D-GS models to boost rendering speed with minimal loss of visual fidelity [1, 2, 6, 7, 10, 11, 18, 19, 25]. Nearly all approaches assign a significance score to each Gaussian that is used to rank and prune them. Several works compute the aggregated ray contribution for each Gaussian across all input images [6, 7, 18, 25], while others combine opacity with additional information, such as gradients per Gaussian, to calculate their pruning criterion [1, 2, 20]. Papantonakis et al. [26] use resolution and scale-aware redundancy metrics, EA-GLES [10] calculates the total transmittance per Gaussian, and Lin et al. [19] accelerate inference speed through an efficiency-aware strategy and foveated rendering.

PUP 3D-GS [11] introduces a more principled approach by deriving a Hessian for each Gaussian that represents its sensitivity to the reconstruction error. While PUP 3D-GS achieves state-of-the-art post-hoc pruning results, computing its sensitivity score incurs considerable storage requirements that limit its viability for use during training. Our pruning approach directly extends PUP 3D-GS by improving its memory efficiency by $36\times$.

2.2. Other Methods

In addition to pruning, several other strategies have been explored to enhance the rendering and training speed of 3D-GS. Mini-Splatting [7] modifies its densification strategies and adds a simplification stage to constrain the number of Gaussians. 3DGS-MCMC [15] models training dynamics as an MCMC process. Revisiting Densification [30] introduces an error-based densification strategy. 3DGS-LM [13] replaces Adam [16] with a tailored Levenberg-Marquardt optimizer [9]. Taming 3DGS [22] proposes a constructive optimization process that limits the number of Gaussians to a pre-defined threshold set by the user. DISTWAR [5] dives into the low-level implementation of GPU thread scheduling and optimizes atomic processing with a novel primitive. StopThePop [27] introduces a precise tile intersection method, but we find that our approach is notably faster in Appendix A.4. FlashGS [8] implements a tile intersection method similar to StopThePop. Several works reduce training time and memory requirements by enforcing geometric constraints [21, 28, 31, 32]. Most of these approaches are orthogonal to ours and can be applied alongside it.

3. Background

3.1. 3D Gaussian Splatting Overview

3D Gaussian Splatting (3D-GS) [14] models scenes as parametric, point-based representations that use differentiable 3D Gaussians as primitives. Given a set of ground

Table 1. Average execution time (milliseconds) of each function across all scenes in Section 5.1. The operation in each row is applied **cumulatively** to all of the following rows. For each model, accurate measurements are collected by averaging execution times across three runs that each render the test set 20 times to reduce variance. The **fastest** and **second** fastest times are color coded.

Method	Preprocess	Inclusive Sum	Duplicate with Keys	Radix Sort	Identify Tile Ranges	Render	Overall
Baseline	0.665	0.045	0.570	1.551	0.082	4.483	7.478
+SnugBox	0.656	0.046	0.208 (2.738×)	0.729 (2.126×)	0.041 (1.980×)	2.344 (1.913×)	4.102 (1.823×)
+AccuTile	0.668	0.046	0.221 (2.575×)	0.612 (2.533×)	0.035 (2.326×)	2.062 (2.175×)	3.748 (1.995×)
+Soft Pruning	0.370 (1.798×)	0.030 (1.494×)	0.146 (3.906×)	0.404 (3.843×)	0.024 (3.422×)	1.337 (3.354×)	2.381 (3.141×)
+Hard Pruning	0.091 (7.293×)	0.016 (2.769×)	0.090 (6.325×)	0.215 (7.217×)	0.013 (6.537×)	0.619 (7.247×)	1.114 (6.712×)

truth training images $\mathcal{I}_{gt} = \{\mathbf{I}_i \in \mathbb{R}^{H \times W}\}_{i=1}^K$, the scene is initialized by using Structure from Motion (SfM) to produce a sparse point cloud that serves as the initial means for the 3D Gaussians. The estimated camera poses $\mathcal{P}_{gt} = \{\phi_i \in \mathbb{R}^{3 \times 4}\}_{i=1}^K$ are paired with their corresponding images and are used to optimize the scene.

Each 3D Gaussian primitive $\mathcal{G}_i$ is parameterized by three geometry parameters – mean $\mu_i \in \mathbb{R}^3$, scale $s_i \in \mathbb{R}^3$, and rotation $r_i \in \mathbb{R}^4$ – and two color parameters – view-dependent spherical harmonics $h_i \in \mathbb{R}^{16 \times 3}$ and opacity $\sigma_i \in \mathbb{R}$. The set of all parameters can be described as:

$$\mathcal{G} = \{\mathcal{G}_i = \{\mu_i, s_i, r_i, h_i, \sigma_i\}\}_{i=1}^N, \quad (1)$$

where N is the number of Gaussians in the model.

Given camera pose ϕ , the scene is rendered by projecting all Gaussians to image space and applying alpha blending to each pixel. Models are optimized via stochastic gradient descent on image reconstruction losses:

$$L(\mathcal{G}|\phi, \mathcal{I}_{gt}) = \|\mathcal{I}_{\mathcal{G}}(\phi) - \mathcal{I}_{gt}\|_1 + L_{D-SSIM}(\mathcal{I}_{\mathcal{G}}(\phi), \mathcal{I}_{gt}), \quad (2)$$

where $\mathcal{I}_{\mathcal{G}}(\phi)$ is the rendered image for pose ϕ .

During optimization, the scene is periodically densified by cloning and splitting uncertain Gaussians and pruned by removing large and transparent Gaussians. The opacities of the Gaussians are also periodically reset.

3.2. 3D Gaussian Splatting Rendering Specifics

3D-GS uses a tile-based rendering strategy that divides the rendered image into 16×16 pixel tiles. Each Gaussian is projected into image space, where its intersection with these tiles is computed. Then, these Gaussian-to-tile mappings are sorted to collect and order the Gaussians by depth for pixel-wise rendering.

Rendering runtime is dominated by six key functions. Table 1 empirically analyzes the execution time of each function and highlights the improvements achieved by our methods. Descriptions of each function are provided in the following sections.

3.2.1. Preprocessing

The `preprocess` kernel is parallelized such that each thread processes a single Gaussian $\mathcal{G}_i$. It computes a 2D

projection of $\mathcal{G}_i$ to image space and obtains a count of tiles that it intersects.

The mean μ_i is projected to image space using a viewing transform $\mathbf{W}$ and a perspective projection, yielding 2D mean $\mu_{i_{2D}}$ and depth that are stored for later processing. The scale s_i and rotation r_i parameters are converted to the diagonal scale $\mathbf{S}_i$ and rotation $\mathbf{R}_i$ matrices. The 3D covariance is then defined as:

$$\Sigma_{i_{3D}} = \mathbf{R}_i \mathbf{S}_i \mathbf{S}_i^T \mathbf{R}_i^T, \quad (3)$$

which is projected via:

$$\hat{\Sigma}_{i_{3D}} = \mathbf{J} \mathbf{W} \Sigma_{i_{3D}} \mathbf{W}^T \mathbf{J}^T, \quad (4)$$

where $\mathbf{J}$ is the Jacobian of the first order approximation of the perspective projection. Dropping the last row and column of $\hat{\Sigma}_{i_{3D}}$ gives $\Sigma_{i_{2D}}$ [14, 33]. The largest eigenvalue of $\Sigma_{i_{2D}}$ is used to compute the count of tiles intersected by this Gaussian $\mathcal{G}_i$ as shown in Figure 2a. $\Sigma_{i_{2D}}^{-1}$ is computed, along with a view-dependent color c_i , derived from $\mathbf{W}$ and h_i . All three values are stored for later processing.

3.2.2. Sorting

After preprocessing, four functions process the Gaussians for pixel-wise rendering:

- **InclusiveSum:** A CUDA primitive that computes the prefix sum over all Gaussian-tile counts to allocate key and value arrays for Gaussian-to-tile mapping.
- **duplicateWithKeys:** A Gaussian-parallel kernel that recomputes Gaussian-tile intersections to generate a key for each intersecting tile, consisting of the tile index and Gaussian depth.
- **RadixSort:** A CUDA primitive that sorts the key array, ordering Gaussian indices by tile and then depth.
- **identifyTileRanges:** A kernel parallelized across the key array to post-process keys before pixel rendering.

3.2.3. Rendering

The `render` kernel is parallelized across pixels. For each pixel p , all Gaussians within its corresponding tile are loaded and processed in depth order as determined by `RadixSort`. An alpha value:

$$\alpha_i(p) = \sigma_i g_i(p) \quad (5)$$

is computed for each Gaussian $\mathcal{G}_i$, where g_i is the value of the projected 2D Gaussian at pixel p :

$$g_i = e^q, q = -\frac{1}{2}(p - \mu_{i_{2D}})\Sigma_{i_{2D}}^{-1}(p - \mu_{i_{2D}})^T. \quad (6)$$

If $\alpha_i > \frac{1}{255}$, then the Gaussian is included in the alpha compositing of the pixel color C , given by:

$$C(p) = \sum_{i \in \mathcal{N}} c_i \alpha_i(p) \prod_{j=1}^{i-1} (1 - \alpha_j(p)). \quad (7)$$

4. Methods

Our Speedy-Splat methods are motivated by two key insights into inefficiencies within the 3D-GS rendering pipeline. First, Gaussian Splatting grossly overestimates the extent of Gaussians in the image. Second, as demonstrated by [11] and other recent pruning works, 3D-GS models are heavily overparameterized.

4.1. Precise Tile Intersect

Gaussian Splatting identifies tiles intersected by Gaussian $\mathcal{G}_i$ by calculating the maximum eigenvalue $\lambda_{\max}$ of its projected 2D covariance $\Sigma_{i_{2D}}$, then selecting all tiles that intersect the square inscribing the circle defined by center $\mu_{i_{2D}}$ and radius:

$$r = \lceil 3\sqrt{\lambda_{\max}} \rceil. \quad (8)$$

This approach neglects opacity σ_i in its calculation and generally overestimates the Gaussian extent, as illustrated by Figure 2a. The actual extent of Gaussian $\mathcal{G}_i$, shown in Figure 2b, is given by the threshold placed on its alpha value α_i . Specifically, $\mathcal{G}_i$ does not contribute to the rendering of pixel p if $\alpha_i < \frac{1}{255}$ for p . By applying the actual extent in tile intersection calculations, we arrive at a far more concise set of intersected tiles.

We now show the derivation of this extent. The furthest pixel extent of Gaussian $\mathcal{G}_i$ can be determined by directly substituting this threshold into Equation 5. Rearranging terms gives:

$$2 \log(255\sigma_i) = (p - \mu_{i_{2D}})\Sigma_{i_{2D}}^{-1}(p - \mu_{i_{2D}})^T. \quad (9)$$

We can rewrite:

$$p = \begin{pmatrix} p_x \\ p_y \end{pmatrix}, \mu_{i_{2D}} = \begin{pmatrix} \mu_x \\ \mu_y \end{pmatrix}, \Sigma_{i_{2D}}^{-1} = \begin{pmatrix} a & b \\ b & c \end{pmatrix}. \quad (10)$$

Specifying threshold t and centered coordinates x_d and y_d :

$$t = 2 \log(255\sigma_i), \quad (11)$$

$$x_d = p_x - \mu_x, \quad (12)$$

$$y_d = p_y - \mu_y, \quad (13)$$

gives the pixel extent with coordinates x_d and y_d that satisfy the ellipse equation:

$$t = ax_d^2 + 2bx_dy_d + cy_d^2. \quad (14)$$

Our approach uses this pixel extent to reduce the number of tiles contained in the Gaussian-to-tile mappings for each Gaussian. We propose two methods for computing precise tile intersections. First, our **SnugBox** algorithm produces a tight bounding box around each Gaussian. Second, our **AccuTile** algorithm extends it to identify the exact set of tiles intersected by the Gaussian.

4.1.1. SnugBox

Our SnugBox method uses this elliptical extent to compute an axis-aligned bounding box that more precisely identifies tiles intersected by Gaussian $\mathcal{G}_i$. To derive this bounding box, we rearrange Equation 14 to solve for y_d :

$$y_d = \frac{-bx_d \pm \sqrt{(b^2 - ac)x_d^2 + tc}}{c}. \quad (15)$$

To find the y -coordinate bounding box edges $y_{\min}$ and $y_{\max}$, we identify the values of x_d where $\partial y_d / \partial x_d = 0$. We refer to these x_d values as $x_{d_{args}}$ to specify that they are the $\arg \min y_d$ and $\arg \max y_d$ values. Differentiating Equation 15 and solving for $x_{d_{args}}$ gives:

$$x_{d_{args}} = \pm \sqrt{\frac{-b^2 t}{(b^2 - ac)a}}. \quad (16)$$

Substituting $x_{d_{args}}$ into Equation 15 and adding μ_y gives $y_{\min}$ and $y_{\max}$. Due to the symmetry of Equation 14, we can find the x -coordinate bounding box edges $x_{\min}$ and $x_{\max}$ by swapping y_d and x_d and constants a and c to rewrite Equations 15 and 16 in terms of x_d and $y_{d_{args}}$.

After the bounding box edges are identified, our method follows 3D-GS and converts these edges to tile indices by dividing by tile size, rounding, and clipping to the image boundary. As depicted in Figure 2, Snugbox can produce a significantly tighter bounding box than 3D-GS. Meanwhile, its computational overhead is small as it performs a constant number of operations and is only called twice in the rendering pipeline – once to count the Gaussians intersecting tiles in preprocess and once to populate the Gaussian-to-tile arrays in `duplicateWithkeys`. Table 1 reports that **SnugBox improves the efficiency of all downstream functions and produces an average overall speed-up of 1.82×**.

4.1.2. AccuTile

Our AccuTile method, outlined in Algorithm 1, extends SnugBox to identify the exact tiles intersected by the Gaussian. It takes as input the tight bounding box produced by Snugbox and its rectangular tile extent – depicted as the

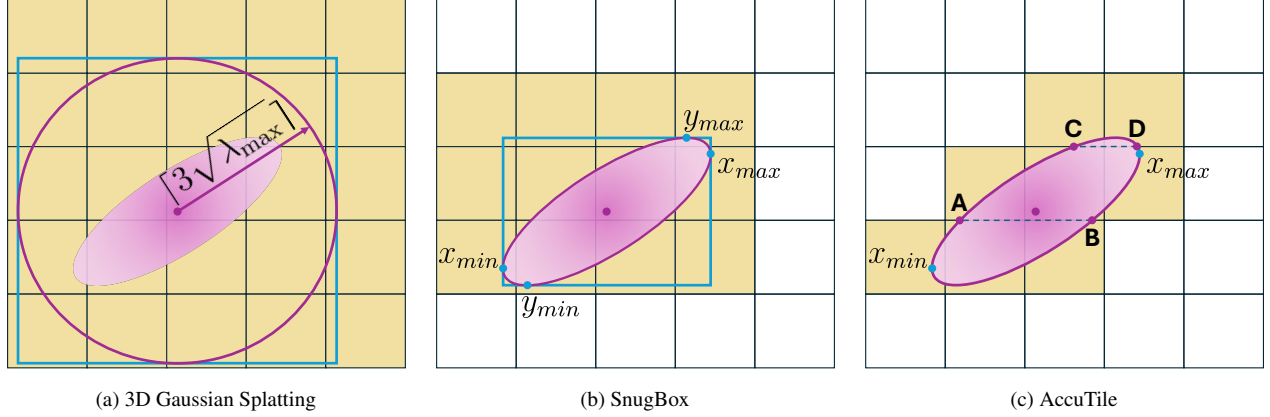


Figure 2. **Gaussian tile allocation by method.** (a) 3D Gaussian Splatting allocates a Gaussian to a tile when that tile **intersects** the **square** inscribing the **circle** with radius $\lceil 3\sqrt{\lambda_{\max}} \rceil$ defined in Equation 8. (b) Our SnugBox method allocates a Gaussian to a tile when that tile **intersects** the tight **bounding box** defined by the axis-aligned minima and maxima of the **ellipse** given by Equation 14. (c) Our AccuTile method allocates a Gaussian to a tile only if that tile **intersects** the **ellipse** via Algorithm 1, which computes the minimum and maximum tiles by iterating over the shorter side of the rectangular tile extent given by SnugBox. In this example, our AccuTile algorithm iterates over the tile rows; the only points that are processed are $x_{\min}$, $x_{\max}$, **A**, **B**, **C**, and **D**.

blue box and yellow tiles, respectively, in Figure 2b. Depending on which dimension is smaller, AccuTile then processes either the rows or columns of this tile extent to determine the exact tiles that intersect the Gaussian. Specifically, we identify the minimum and maximum extent of the ellipse within a given row or column, then convert those points to the corresponding touched tiles. All tiles between the minimum and maximum tiles intersect the ellipse.

AccuTile’s key insight is that calculating the minimum and maximum extent of the ellipse within each row or column requires computing only two points per iteration. The only possible inflection points of the elliptical curve are the bounding box minimum and maximum points identified by Snugbox. If one or both of these points lie within the tile row or column, then they represent the minimum or maximum extent of the ellipse there. If neither point is within the row or column, then the points along that boundary side are monotonically decreasing or increasing – the minimum or maximum point must lie on one of the boundary lines of that row or column. Since the boundary of the last row or column is shared with the next one, we only need to compute the intersection of the ellipse with the next boundary line in each iteration. Thus, our AccuTile algorithm counts tiles in time proportional to the shorter side of the tile extent and processes tiles in time proportional to the tile count.

Figure 2 illustrates how Accutile restricts the tight **bounding box** produced by SnugBox to the exact tiles that the Gaussian touches. The **tile extent rows** from Snugbox, shown in Figure 2b, are processed starting from the bottom. The **lower tile row boundary** falls below the **bounding box**, so no initial intersection is computed. The **upper boundary** lies below the top of the **bounding box**, so intersection

points **A** and **B** are calculated using Equation 15. $x_{\min}$ is assigned as this row’s minimum ellipse extent $e_{\min}$ because it lies within it, and **B** is assigned as its maximum ellipse extent $e_{\max}$ because it is the maximal point. Consequently, this row’s **tile extent** is from the tile containing $x_{\min}$ to the tile containing **B**. For the next row, we keep points **A** and **B** and compute upper boundary points **C** and **D**. **A** and $x_{\max}$ are assigned as the row’s $e_{\min}$ and $e_{\max}$, and the tiles containing them are its **tile extent**. Finally, for the last row, **C** and **D** are kept; no additional points are computed because the row’s **upper boundary** is above the **bounding box**. Thus, the **tile extent** of this row is between **C** and **D**.

The number of tiles is further reduced from Figure 2b to Figure 2c, much less than the original 3D-GS method in Figure 2a. In Table 1, we report that AccuTile further accelerates all downstream functions, culminating in an average overall speed-up of $1.99\times$.

4.2. Efficient Pruning

PUP 3D-GS [11] is a pruning method that quantifies the sensitivity of each Gaussian to training views, removing a set percentage with the lowest sensitivities. Sensitivity is computed by approximating the Hessian of the L_2 loss:

$$H = \nabla_{\mathcal{G}}^2 L_2 = \sum_{\phi \in \mathcal{P}_{gt}} \nabla_{\mathcal{G}} I_{\mathcal{G}}(\phi) \nabla_{\mathcal{G}} I_{\mathcal{G}}(\phi)^T, \quad (17)$$

where $\nabla_{\mathcal{G}} I_{\mathcal{G}}(\phi)$ is the gradient over all Gaussian parameters on the rendered image $I_{\mathcal{G}}$ for pose ϕ . H is shown to be exact when the L_1 residual error vanishes [11].

A per-Gaussian sensitivity can be derived by splitting H into the block diagonal elements that only capture inter-

Algorithm 1 The AccuTile Algorithm. For simplicity, the algorithm outlined here is applied to the rows of the SnugBox tile extent bounding box, matching the example in Figure 2c. In practice, it is applied along the smaller side of the tile extent. The subscripts t , b , l , and r represent the *top*, *bottom*, *left*, and *right* sides, respectively. A proof of correctness sketch is presented in Appendix A.1.

Require: Ellipse **E** ▷ Eq. 14
Require: SnugBox Bounding Box **B**
Require: SnugBox Tile Extent Rectangle **R**
Initialize: Tile count **C** $\leftarrow 0$

```

line_min  $\leftarrow \mathbf{R}_b$ 
if line_min  $\geq \mathbf{B}_b$  then
    i_min  $\leftarrow \text{Intersections}(\text{line\_min}, \mathbf{E})$  ▷ Eq. 15
end if
for row r in R do
    line_max  $\leftarrow \mathbf{r}_t$ 
    if line_max  $\leq \mathbf{B}_t$  then
        i_max  $\leftarrow \text{Intersections}(\text{line\_max}, \mathbf{E})$  ▷ Eq. 15
    end if
    e_min  $\leftarrow \mathbf{B}_l$  if  $\mathbf{B}_l$  in r else min(i_min, i_max)
    e_max  $\leftarrow \mathbf{B}_r$  if  $\mathbf{B}_r$  in r else max(i_min, i_max)
    tile_min, tile_max  $\leftarrow \text{Convert}(\mathbf{e\_min}, \mathbf{e\_max})$ 
    C  $\leftarrow \mathbf{C} + (\text{tile\_max} - \text{tile\_min})$ 
    Process(tile_min, tile_max)
    i_min  $\leftarrow \mathbf{i\_max}$ 
end for
return C

```

Gaussian parameter relationships:

$$H_i = \sum_{\phi \in \mathcal{P}_{gt}} \nabla_{\mathcal{G}_i} I_{\mathcal{G}}(\phi) \nabla_{\mathcal{G}_i} I_{\mathcal{G}}(\phi)^T, \quad (18)$$

where $\nabla_{\mathcal{G}_i}$ is the gradient with respect to only $\mathcal{G}_i$. This measures the sensitivity of the L_2 loss with respect to Gaussian $\mathcal{G}_i$, assuming all other Gaussians are held constant.

H_i is again approximated by only using the six mean μ_i and scale s_i parameters to specifically capture geometric sensitivity. The log determinant is taken to provide a representative scalar score U_i :

$$U_i = \log |\nabla_{\mu_i, s_i} I_{\mathcal{G}} \nabla_{\mu_i, s_i} I_{\mathcal{G}}^T|. \quad (19)$$

Using this score, up to 90% of Gaussians can be robustly pruned from the model while retaining high visual quality.

Although PUP 3D-GS touts high compression ratios and rendering speeds, we identify two key drawbacks in its formulation. First, computing the Hessian requires storage

proportional to $N \times 36$, where N is the number of Gaussians. In comparison, the 3D-GS model has a memory footprint proportional to $N \times 59$ because it stores 59 parameters per Gaussian. While this score is effective for post-hoc pruning, using it during training is impractical.

Second, computing the Hessian requires the pixel-wise gradients of μ and s . Since these are 3D parameters of the Gaussian primitives, obtaining their gradients requires back-propagating through the `render` kernel parallelized per pixel, then back-propagating each Gaussian contributing to that pixel in its thread. This breaks the efficient flow of gradients in 3D-GS, where the per-pixel gradients from the `render` kernel are parallelized and aggregated to the 2D μ_{2D} and Σ_{2D} parameters, which are then parallelized across Gaussians to compute gradients for μ and s .

Our approach builds on PUP 3D-GS by introducing an **efficient pruning score** that we incorporate into the 3D-GS training pipeline. We also define two distinct pruning modalities: **Soft Pruning**, which takes place during densification in the first 15000 iterations, and **Hard Pruning**, which is applied after densification is completed after iteration 15000.

4.2.1. Efficient Pruning Score

Our insight is that both drawbacks can be alleviated by reparameterizing the Hessian. Concretely, we express the influence of all spatial parameters of Gaussian $\mathcal{G}_i$ by computing the Hessian approximation with respect to the 2D projected value of $\mathcal{G}_i$ at pixel p , given by $g_i(p)$ in Equation 6. In doing so, the pruning score U_i from Equation 19 can be expressed as:

$$\tilde{U}_i = \log |\nabla_{g_i} I_{\mathcal{G}} \nabla_{g_i} I_{\mathcal{G}}^T|. \quad (20)$$

Since g_i is a scalar and log is monotonically increasing, we can rewrite this score as:

$$\tilde{U}_i = (\nabla_{g_i} I_{\mathcal{G}})^2. \quad (21)$$

Gradient $\nabla_{g_i} I_{\mathcal{G}}$ is already computed in the backward pass of `render` and can be efficiently squared and aggregated across all pixels. Moreover, the maximum space requirement for this score is proportional to the number of Gaussians N , reducing the storage requirement by $36\times$ and allowing this score to be used during training.

4.2.2. Soft Pruning

To maintain a robust Hessian approximation, we observe that the L_1 loss becomes quite small by iteration 6000 and remains low except after an opacity reset is performed. As such, we augment the densification pipeline to include our Soft Pruning method, where the model is pruned immediately before the three opacity resets at 6000, 9000, and 12000 iterations. Surprisingly, we find that we can set extremely high Soft Pruning ratios – in our experiments, visual fidelity is preserved at 80% pruning.

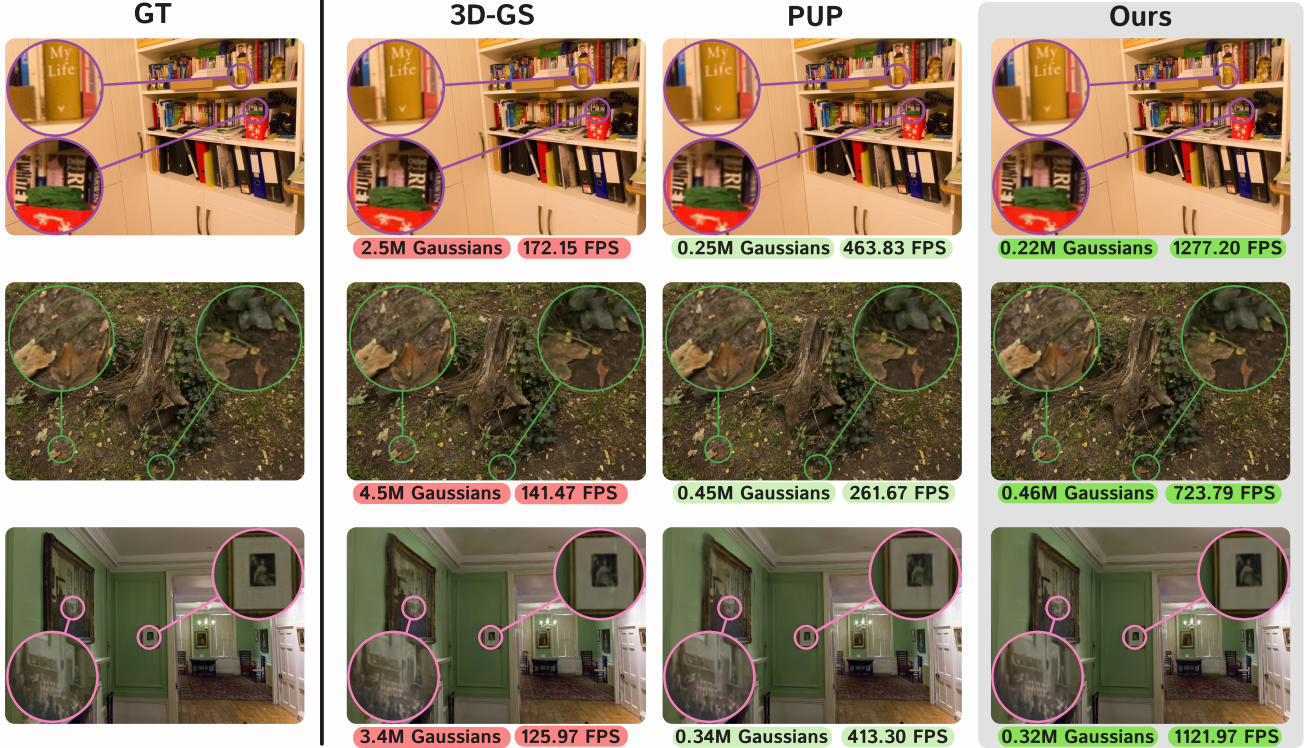


Figure 3. Visual comparison of 3D-GS, PUP 3D-GS, and our method. Notice that, while reaching similar compression ratios to PUP 3D-GS, our Speedy-Splat method delivers vastly faster rendering speeds. Top: *playroom* from the Deep Blending dataset. Middle: *bicycle* from the Mip-NeRF 360 dataset. Bottom: *drjohnson* from the Deep Blending dataset.

4.2.3. Hard Pruning

We also observe that the model’s performance after densification closely matches that of the fully-trained model. The iterations after the densification stage essentially fine-tune the model and can be used to further “refine” it after pruning, similar to PUP 3D-GS [11] and LightGaussian [6]. In practice, our Hard Pruning method prunes the model by a constant ratio every 3000 iterations starting at iteration 15000. We Hard Prune 30% of Gaussians in each interval, which, when paired with Soft Pruning, empirically reduces the total number of Gaussians across scenes by 10.6 $\times$.

5. Experiments

5.1. Datasets

Our evaluation uses the same set of challenging real-world scenes as 3D-GS [14]. This includes nine Mip-Nerf 360 scenes [3] – four indoor and five outdoor – that each feature a complex central object or area with a detailed background. We also include the outdoor *train* and *truck* scenes from the Tanks & Temples dataset [17] and the indoor *drjohnson* and *playroom* scenes from the Deep Blending dataset [12]. For consistency across experiments, we use the COLMAP pose estimates and sparse point clouds provided by the dataset authors.

Table 2. Average Gaussian count, FPS, and training time across all scenes in Section 5.1. Ratios for model size compression, rendering speed-up, and training speed-up are reported in (parentheses). The operation in each row is applied **cumulatively** to all of the following rows. The **best** and **second** best value for each metric are color coded.

Method	# Gaussians ↓	FPS ↑	Training Time ↓
Baseline	2.93M	134	23.2
+SnugBox	2.97M	244 (1.82 $\times$)	21.2 (1.09 $\times$)
+AccuTile	2.97M	267 (1.99 $\times$)	21.0 (1.10 $\times$)
+Soft Pruning	1.64M (1.79 $\times$)	420 (3.14 $\times$)	17.5 (1.32 $\times$)
+Hard Pruning	0.28M (10.6 $\times$)	898 (6.71 $\times$)	15.7 (1.47 $\times$)

5.2. Implementation Details

Our code builds on the differentiable renderer provided by 3D-GS [14] and modifies the Python training pipeline for pruning schedules and execution. To ensure consistent and precise timing, all times in Table 1 and FPS values in Tables 2 and 3 are measured using CUDA events at the start and end of the forward rendering procedure. All experiments are conducted on an Nvidia RTX A5000 GPU, and the reported Speedy-Splat results represent the average metrics across three independent runs for each scene.

Table 3. Average reported metrics for each pruning method across all scenes in the Mip-NeRF 360 dataset. The *Comp* column reports model size compression in terms of Gaussian count, *FPS* reports rendering speed-up, and *Train* reports training time speed-up, all with respect to the baseline 3D-GS model. PSNR, SSIM, and LPIPS are also recorded. For a fair comparison, we report the published results of each method and use ‘-’ to denote missing metrics. The **best** and **second** best value for each metric are color coded; lossless methods are underlined. Results for the Tanks & Temples and Deep Blending datasets are reported in Appendix A.3.

Method	Comp $\uparrow$	FPS $\uparrow$	Train $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
3D-GS [14]	1.00 $\times$	1.00 $\times$	1.00 $\times$	27.55	0.814	0.222
Trimming [2]	4.00 $\times$	-	-	27.13	0.798	0.248
Compact [18]	2.28 $\times$	1.07 $\times$	0.73 $\times$	27.08	0.798	0.247
EAGLES [10]	3.68 $\times$	1.51 $\times$	1.37$\times$	26.94	0.800	0.250
Reducing [26]	2.33 $\times$	1.60 $\times$	1.23 $\times$	27.10	0.809	0.226
Light [6]	2.94 $\times$	1.76 $\times$	-	27.28	0.805	0.243
ELMGS [1]	5.00 $\times$	2.69 $\times$	-	27.00	0.779	0.286
PUP [11]	8.65$\times$	2.55 $\times$	-	26.83	0.792	0.268
Mini-Splat [7]	6.84 $\times$	3.20$\times$	1.26 $\times$	27.34	0.822	0.217
+SnugBox	0.99 $\times$	1.81 $\times$	1.08 $\times$	27.55	0.814	0.221
+AccuTile	0.99 $\times$	1.99 $\times$	1.10 $\times$	27.57	0.814	0.221
+Soft Pruning	1.79 $\times$	3.14 $\times$	1.30 $\times$	27.32	0.807	0.246
+Hard Pruning	10.6$\times$	6.51$\times$	1.45$\times$	26.94	0.782	0.296

5.3. Results

5.3.1. Additive method performance

The efficacy of Speedy-Splat is demonstrated by Table 1, where we record the average execution times of each function across all scenes when additively applying our methods. SnugBox and AccuTile each introduce minimal additional computation time to preprocess and InclusiveSum. However, limiting the number of tiles touched accelerates all downstream functions, culminating in an overall speed-up of 1.82 $\times$ by SnugBox that is raised to 1.99 $\times$ by AccuTile. Applying soft pruning reduces the runtime of all functions by reducing the number of Gaussians, leading to a 3.14 $\times$ overall speed-up. Finally, performing hard pruning improves overall speed by a whopping 6.71 $\times$ over the baseline 3D-GS model.

5.3.2. Overall Performance

In Figures 1 and 3, we report qualitative results on two outdoor and two indoor scenes from all three datasets. The magnified regions highlight that our method preserves the fine details in the baseline 3D-GS scene and closely models the ground truth view. Despite touting similar compression ratios and rendering nearly identical images, Speedy-Splat achieves over double the FPS of PUP 3D-GS.

Table 3 compares our methods against other methods that reduce the number of Gaussians and increase inference speed using the mean of each metric across all scenes in the Mip-NeRF 360 dataset. The underlined methods are “lossless”, meaning that they avoid degrading visual fidelity at all. SnugBox and AccuTile, our lossless methods, improve

Table 4. Average metrics across all scenes in Section 5.1 when using the Speedy-Splat and PUP 3D-GS pruning scores to prune 88.44% of Gaussians using the PUP 3D-GS pipeline. Two rounds of prune-refine are performed on each baseline 3D-GS model, pruning 66% of Gaussians and then fine-tuning for 5,000 iterations in each one. The **best** value for each metric is color coded.

Method	# Gaussians $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$
Baseline	2.92M	27.1503	0.8296	0.2238	107.53
PUP [11]	0.34M	26.2136	0.8044	0.2731	378.57
Ours	0.34M	26.8658	0.8022	0.2840	345.52

rendering and training speed while leaving image quality metrics essentially unchanged or slightly better. Our full pipeline, labeled as “+Hard Pruning” boasts the highest compression ratios, rendering speeds, and training speed-ups across all datasets. Furthermore, its image quality metrics are competitive with the other methods.

5.4. Pruning Score Comparison

Although the primary focus of our work is rendering speed, we find that our efficient pruning score, described in Section 4.2, also performs well when applied in other compression pipelines. In Table 4, we ablate our efficient pruning score with the PUP 3D-GS sensitivity score across all scenes in their post-hoc pruning pipeline. Notably, Speedy-Splat’s efficient pruning score outperforms PUP 3D-GS on PSNR and is competitive across the other metrics.

6. Limitations

A limitation of Speedy-Splat is that it produces slightly lower image quality than 3D-GS. However, this degradation is expected at high compression ratios and is also observed in comparable techniques. Additionally, a direct comparison of our efficient pruning score to the PUP 3D-GS pruning score illuminates a slight, yet noticeable, gap in performance. Future work could explore the possibility of another efficient pruning score that delivers higher performance.

7. Conclusion

In this work, we present Speedy-Splat: a new 3D-GS technique that accurately localizes Gaussians during rendering and significantly improves inference speed, model size, and training time. Enhanced localization is achieved by our SnugBox and AccuTile methods, while model size reduction is accomplished by our Soft and Hard Pruning approaches. Together, our Speedy-Splat methods accelerate rendering speed by an average of 6.71 $\times$, reduce model size by 10.6 $\times$, and improve training time by 1.47 $\times$ across all scenes from the Mip-NeRF 360, Tanks & Temples, and Deep Blending datasets.

8. Acknowledgements

This work was made possible by the IARPA WRIVA Program, the ONR MURI program, and DAPRA TIAMAT. Commercial support was provided by Capital One Bank, the Amazon Research Award program, and Open Philanthropy. Further support was provided by the National Science Foundation (IIS-2212182), and by the NSF TRAILS Institute (2229885). Zwicker was additionally supported by the National Science Foundation (IIS-2126407).

References

- [1] Muhammad Salman Ali, Sung-Ho Bae, and Enzo Tartaglione. ElmgS: Enhancing memory and computation scalability through compression for 3d gaussian splatting. *arXiv preprint arXiv:2410.23213*, 2024. 2, 8, 12
- [2] Muhammad Salman Ali, Maryam Qamar, Sung-Ho Bae, and Enzo Tartaglione. Trimming the fat: Efficient compression of 3d gaussian splats through pruning. *arXiv preprint arXiv:2406.18214*, 2024. 2, 8, 12
- [3] Jonathan T Barron, Ben Mildenhall, Dor Verbin, Pratul P Srinivasan, and Peter Hedman. Mip-nerf 360: Unbounded anti-aliased neural radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5470–5479, 2022. 7
- [4] Anpei Chen, Zexiang Xu, Andreas Geiger, Jingyi Yu, and Hao Su. Tensorf: Tensorial radiance fields. In *European Conference on Computer Vision (ECCV)*, 2022. 1
- [5] Sankeerth Durvasula, Adrian Zhao, Fan Chen, Ruofan Liang, Pawan Kumar Sanjaya, and Nandita Vijaykumar. Distwar: Fast differentiable rendering on raster-based rendering pipelines. *arXiv preprint arXiv:2401.05345*, 2023. 2
- [6] Zhiwen Fan, Kevin Wang, Kairun Wen, Zehao Zhu, De-jia Xu, and Zhangyang Wang. Lightgaussian: Unbounded 3d gaussian compression with 15x reduction and 200+ fps. *arXiv preprint arXiv:2311.17245*, 2023. 1, 2, 7, 8, 12
- [7] Guangchi Fang and Bing Wang. Mini-splatting: Representing scenes with a constrained number of gaussians. *arXiv preprint arXiv:2403.14166*, 2024. 1, 2, 8, 12
- [8] Guofeng Feng, Siyan Chen, Rong Fu, Zimu Liao, Yi Wang, Tao Liu, Zhilin Pei, Hengjie Li, Xingcheng Zhang, and Bo Dai. Flashgs: Efficient 3d gaussian splatting for large-scale and high-resolution rendering. *arXiv preprint arXiv:2408.07967*, 2024. 1, 2
- [9] Henri P. Gavin. The levenberg-marquardt method for non-linear least squares curve-fitting problems c ©. 2013. 2
- [10] Sharath Girish, Kamal Gupta, and Abhinav Shrivastava. Eagles: Efficient accelerated 3d gaussians with lightweight encodings. *arXiv preprint arXiv:2312.04564*, 2023. 2, 8, 12
- [11] Alex Hanson, Allen Tu, Vasu Singla, Mayuka Jayawardhana, Matthias Zwicker, and Tom Goldstein. Pup 3d-gs: Principled uncertainty pruning for 3d gaussian splatting. *arXiv preprint arXiv:2406.10219*, 2024. 1, 2, 4, 5, 7, 8, 12
- [12] Peter Hedman, Julien Philip, True Price, Jan-Michael Frahm, George Drettakis, and Gabriel Brostow. Deep blending for free-viewpoint image-based rendering. *ACM Transactions on Graphics*, 37(6):1–15, 2018. 7
- [13] Lukas Höllein, Aljaž Božič, Michael Zollhöfer, and Matthias Nießner. 3dgs-lm: Faster gaussian-splatting optimization with levenberg-marquardt. *arXiv preprint arXiv:2409.12892*, 2024. 2
- [14] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4):1–14, 2023. 1, 2, 3, 7, 8, 12, 13
- [15] Shakiba Kheradmand, Daniel Rebain, Gopal Sharma, Weiwei Sun, Yang-Che Tseng, Hossam Isack, Abhishek Kar, Andrea Tagliasacchi, and Kwang Moo Yi. 3d gaussian splatting as markov chain monte carlo. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. Spotlight Presentation. 2
- [16] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *CoRR*, abs/1412.6980, 2014. 2
- [17] Arno Knapitsch, Jaesik Park, Qian-Yi Zhou, and Vladlen Koltun. Tanks and temples: Benchmarking large-scale scene reconstruction. *ACM Transactions on Graphics*, 36(4):1–13, 2017. 7
- [18] Joo Chan Lee, Daniel Rho, Xiangyu Sun, Jong Hwan Ko, and Eunbyung Park. Compact 3d gaussian representation for radiance field. *arXiv preprint arXiv:2311.13681*, 2023. 2, 8, 12
- [19] Weikai Lin, Yu Feng, and Yuhao Zhu. RtgS: Enabling real-time gaussian splatting on mobile devices using efficiency-guided pruning and foveated rendering. *arXiv preprint arXiv:2407.00435*, 2024. 1, 2
- [20] Xiangrui Liu, Xinju Wu, Pingping Zhang, Shiqi Wang, Zhu Li, and Sam Kwong. CompGS: Efficient 3d scene representation via compressed gaussian splatting. *arXiv preprint arXiv:2404.09458*, 2024. 2
- [21] Tao Lu, Mulin Yu, Linning Xu, Yuanbo Xiangli, Limin Wang, Dahua Lin, and Bo Dai. Scaffold-gs: Structured 3d gaussians for view-adaptive rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20654–20664, 2024. 2
- [22] Saswat Subhajyoti Mallick, Rahul Goel, Bernhard Kerbl, Markus Steinberger, Francisco Vicente Carrasco, and Fernando De La Torre. Taming 3dgs: High-quality radiance fields with limited resources. In *SIGGRAPH Asia 2024 Conference Papers*, New York, NY, USA, 2024. Association for Computing Machinery. 2
- [23] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021. 1
- [24] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Trans. Graph.*, 41(4):102:1–102:15, 2022. 1
- [25] Michael Niemeyer, Fabian Manhardt, Marie-Julie Rakotsasona, Michael Oechsle, Daniel Duckworth, Rama Gosula, Keisuke Tateno, John Bates, Dominik Kaeser, and Federico Tombari. Radsplat: Radiance field-informed gaussian splatting for robust real-time rendering with 900+ fps. *arXiv preprint arXiv:2403.13806*, 2024. 1, 2

- [26] Panagiotis Papantonakis, Georgios Kopanas, Bernhard Kerbl, Alexandre Lanvin, and George Drettakis. Reducing the memory footprint of 3d gaussian splatting. *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, 7(1):1–17, 2024. [2](#), [8](#), [12](#)
- [27] Lukas Radl, Michael Steiner, Mathias Parger, Alexander Weinrauch, Bernhard Kerbl, and Markus Steinberger. StopThePop: Sorted Gaussian Splatting for View-Consistent Real-time Rendering. *ACM Transactions on Graphics*, 4(43), 2024. [2](#), [13](#)
- [28] Kerui Ren, Lihan Jiang, Tao Lu, Mulin Yu, Linning Xu, Zhangkai Ni, and Bo Dai. Octree-gs: Towards consistent real-time rendering with lod-structured 3d gaussians. *arXiv preprint arXiv:2403.17898*, 2024. [2](#)
- [29] Fernando Rivas-Manzanique, Jorge Sierra-Acosta, Adrian Penate-Sanchez, Francesc Moreno-Noguer, and Angela Ribeiro. Nerflight: Fast and light neural radiance fields using a shared feature grid. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12417–12427, 2023. [1](#)
- [30] Samuel Rota Bulò, Lorenzo Porzi, and Peter Kotschieder. Revising densification in gaussian splatting. In *European Conference on Computer Vision*, pages 347–362. Springer, 2024. [2](#)
- [31] Evangelos Ververas, Rolandos Alexandros Potamias, Jifei Song, Jiankang Deng, and Stefanos Zafeiriou. Sags: structure-aware 3d gaussian splatting. In *European Conference on Computer Vision*, pages 221–238. Springer, 2024. [2](#)
- [32] Meng Wei, Qianyi Wu, Jianmin Zheng, Hamid Rezatofighi, and Jianfei Cai. Normal-gs: 3d gaussian splatting with normal-involved rendering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. [2](#)
- [33] Matthias Zwicker, Hanspeter Pfister, Jeroen Van Baar, and Markus Gross. Ewa splatting. *IEEE Transactions on Visualization and Computer Graphics*, 8(3):223–238, 2002. [3](#)

A. Appendix

A.1. AccuTile Proof of Correctness Sketch

We outline the correctness of our AccuTile algorithm by examining the different cases that arise when identifying the minimum and maximum points of an ellipse within a given tile row. Due to the symmetry of Equation 14, exchanging the variables x and y along with the coefficients a and c yields equivalent statements for tile columns. Thus, we focus our discussion on tile rows.

Case 1: The ellipse does not intersect the tile row boundary. The entire ellipse, including the bounding box extrema $x_{\min}$ and $x_{\max}$ computed by SnugBox, lies within the row. AccuTile correctly selects these points as the furthest ellipse extents. Figure 4 illustrates an example of this.

Case 2: The ellipse intersects one of the tile row bounding lines but not the other. This implies that either $y_{\min}$ or $y_{\max}$ lies within the row, but not both. There are several possible subcases:

- **Case 2.1:** If both $x_{\min}$ and $x_{\max}$ are in the tile row, then they are correctly assigned as the furthest extent of the ellipse by AccuTile. Figure 5 illustrates an example of this case.
- **Case 2.2:** If $x_{\min}$ and $x_{\max}$ are not in the row but $y_{\max}$ is, then the ellipse decreases monotonically from $y_{\max}$ to the row boundary on both sides of $y_{\max}$, making the row boundary intersections the furthest extent of the ellipse and are the points selected by AccuTile as the furthest row extent. This follows from the absence of the critical points $x_{\min}$ and $x_{\max}$. A symmetric argument applies when $y_{\min}$ is in the row instead. The top tile row of Figures 5 and 6 illustrate examples of this case.
- **Case 2.3:** If $x_{\min}$ and $y_{\min}$ are in the row but $x_{\max}$ is not, then $x_{\min}$ is assigned as the minimum extent of the ellipse. The ellipse increases monotonically from $y_{\min}$ to the boundary to the right of $y_{\min}$ and from $x_{\min}$ to the boundary to the right of $x_{\min}$. Under a corollary of the definition of an ellipse, the sides of the ellipse do not intersect. Thus, the ellipse point on the curve that extends to the right of $y_{\min}$ and intersects the tile row boundary must be the maximum ellipse extent, and AccuTile correctly selects it as such. A similar argument applies in the following cases: (1) $x_{\max}$ and $y_{\min}$ are in the tile row but $x_{\min}$ is not, (2) $x_{\min}$ and $y_{\max}$ are in the tile row but $x_{\max}$ is not, and (3) $x_{\max}$ and $y_{\max}$ are in the tile row but $x_{\min}$ is not. The bottom tile row of Figure 6 illustrates an example of this case.

Case 3: The ellipse intersects both the top and bottom row boundary.

If $x_{\min}$ or $x_{\max}$ is in the tile row, then AccuTile correctly assigns it as the minimum or maximum extent of the ellipse, respectively. The right side of the ellipse in the middle tile

row in Figure 6 illustrates an example of this case. Otherwise, the ellipse monotonically increases from the bottom row boundary to the top row boundary, or vice-versa, due to the absence of critical points. Selecting the minimum or maximum boundary point, as done by AccuTile, yields the correct result. The left side of the ellipse in the middle tile row in Figure 6 illustrates an example of this case.

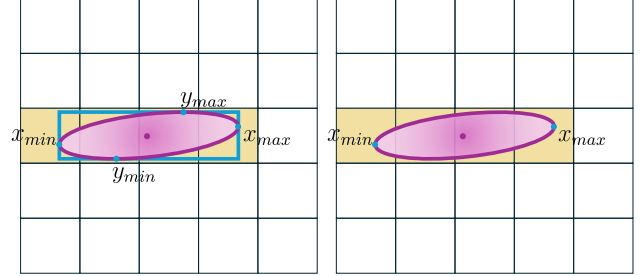


Figure 4. (Left) SnugBox and (right) AccuTile sketch of Case 1. As with Figure 2c, our AccuTile algorithm iterates over the tile rows; the only points that are processed are $x_{\min}$ and $x_{\max}$.

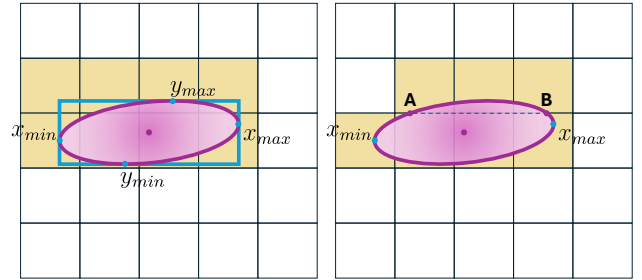


Figure 5. (Left) SnugBox and (right) AccuTile sketch of Case 2.1. Our AccuTile algorithm iterates over the tile rows; the only points that are processed are $x_{\min}$, $x_{\max}$, A, and B.

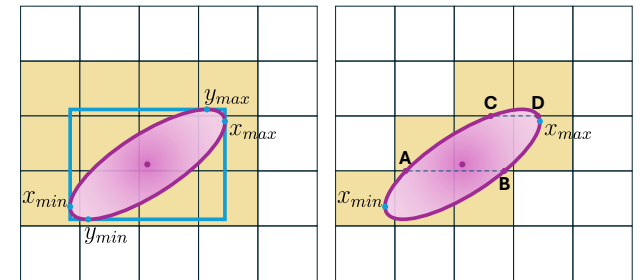


Figure 6. (Left) SnugBox and (right) AccuTile sketch of Cases 2.2, 2.3, and 3. Our AccuTile algorithm iterates over the tile rows; the only points that are processed are $x_{\min}$, $x_{\max}$, A, B, C, and D. A detailed walkthrough of this example is presented in Section 4.1.2.

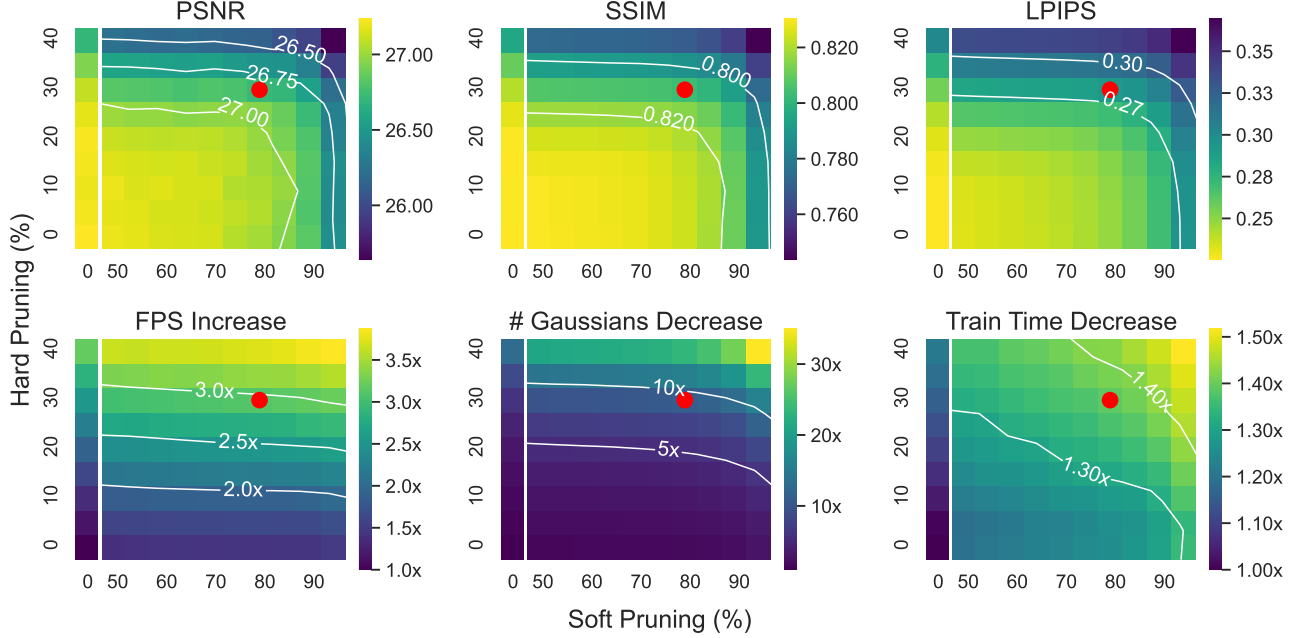


Figure 7. We sweep pruning percentages in 5% increments for Hard Pruning (0% – 40%) and Soft Pruning (0%, 50% – 95%) on all scenes listed in Section 5.1. Experiments are performed 3 $\times$ on each scene without our Gaussian localization methods; the reported metrics are averaged across all runs. (0%, 0%) is the baseline 3D-GS model, the first column (0%, :) is Hard Pruning in isolation, and the first row (:, 0%) is Soft Pruning in isolation. The red dots at (80%, 30%) denote the percentage settings used in our manuscript. We report the FPS increase and the Number of Gaussians and Train Time decrease factors to be consistent with the format in Table 3.

Table 5. Average reported metrics for each pruning method across all scenes in the Tanks & Temples dataset.

Method	Comp $\uparrow$	FPS $\uparrow$	Train $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
3D-GS [14]	1.00 $\times$	1.00 $\times$	1.00 $\times$	23.70	0.849	0.178
Trimming [2]	4.00 $\times$	-	-	23.69	0.831	0.210
Compact [18]	2.19 $\times$	1.16 $\times$	0.76 $\times$	23.32	0.831	0.201
EAGLES [10]	-	1.73 $\times$	1.19 $\times$	23.10	0.820	0.220
Reducing [26]	2.56 $\times$	1.91 $\times$	1.27 $\times$	23.57	0.840	0.188
Light [6]	2.94 $\times$	1.97 $\times$	-	23.11	0.817	0.231
ELMGS [1]	5.00 $\times$	4.05 $\times$	-	23.90	0.825	0.233
PUP [11]	10.0 $\times$	4.00 $\times$	-	22.72	0.801	0.244
Mini-Splat [7]	9.20 $\times$	-	-	23.18	0.835	0.202
+SnugBox	0.99 $\times$	1.61 $\times$	1.11 $\times$	23.69	0.849	0.178
+AccuTile	0.99 $\times$	1.67 $\times$	1.12 $\times$	23.73	0.849	0.177
+Soft Pruning	1.69 $\times$	2.48 $\times$	1.36 $\times$	23.54	0.841	0.201
+Hard Pruning	10.1 $\times$	6.30 $\times$	1.58 $\times$	23.45	0.821	0.241

A.2. Overall Pruning Percent Metrics

In Figure 7, we perform a parameter sweep over Hard Pruning percentages from 0% – 40% at 5% intervals and Soft Pruning percentages at 0% and from 50 – 95% at 5% intervals. We conduct each experiment 3 $\times$ on each scene listed in Section 5.1 to reduce variance, then average the metrics across all runs. All experiments are run without our Gaussian localization methods – SnugBox and AccuTile – to ablate the effect of each pruning method in isolation. Our (80%, 30%) pruning percentages are empirically selected to produce a favorable balance between speed and quality.

Table 6. Average reported metrics for each pruning method across all scenes in the Deep Blending dataset.

Method	Comp $\uparrow$	FPS $\uparrow$	Train $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
3D-GS [14]	1.00 $\times$	1.00 $\times$	1.00 $\times$	29.09	0.886	0.288
Trimming [2]	1.33 $\times$	-	-	29.43	0.897	0.267
Compact [18]	2.65 $\times$	1.37 $\times$	0.79 $\times$	29.79	0.901	0.258
EAGLES [10]	-	1.30 $\times$	1.31 $\times$	29.92	0.900	0.250
Reducing [26]	2.86 $\times$	1.79 $\times$	1.27 $\times$	29.63	0.902	0.249
Light [6]	-	-	-	-	-	-
ELMGS [1]	5.00 $\times$	4.15 $\times$	-	29.24	0.894	0.273
PUP [11]	10.0 $\times$	4.51 $\times$	-	28.85	0.881	0.301
Mini-Splat [7]	8.06 $\times$	-	-	29.98	0.908	0.253
+SnugBox	0.97 $\times$	2.11 $\times$	1.12 $\times$	29.18	0.886	0.287
+AccuTile	0.97 $\times$	2.32 $\times$	1.13 $\times$	29.12	0.885	0.288
+Soft Pruning	1.86 $\times$	3.56 $\times$	1.41 $\times$	29.29	0.889	0.296
+Hard Pruning	11.1 $\times$	7.46 $\times$	1.57 $\times$	29.32	0.887	0.311

A.3. Additional Datasets Evaluation

Table 5 and Table 6 present the average reported metrics for each pruning method across all scenes in the Tanks & Temples and Deep Blending datasets, respectively. The *Comp* column reports model size compression in terms of Gaussian count, *FPS* reports rendering speed-up, and *Train* reports training time speed-up, all with respect to the baseline 3D-GS model. PSNR, SSIM, and LPIPS are also recorded. The **best** and **second best** value for each metric are color coded; lossless methods are underlined.

Table 7. Average execution time (milliseconds) of each function across all scenes. This experiment ablates the StopThePop [27] Tile-Based Culling method with warp-level load balancing against our AccuTile algorithm. For each method, execution times are averaged over three runs, with each run rendering the test set 20 times to reduce variance. The **fastest** times are highlighted. Our AccuTile algorithm outperforms the Tile-Based Culling method in overall runtime by a notable margin. For detailed analysis, see Section A.4.

Method	Preprocess	Inclusive Sum	Duplicate with Keys	Radix Sort	Identify Tile Ranges	Render	Overall
Baseline	0.665	0.046	0.568	1.551	0.083	4.469	7.457
Tile-Based Culling [27]	0.811 (0.820x)	0.046	0.450 (1.263x)	0.609 (2.548x)	0.035 (2.341x)	2.027 (2.205x)	4.051 (1.841x)
AccuTile (Ours)	0.659	0.046	0.194 (2.931x)	0.610 (2.541x)	0.035 (2.338x)	2.042 (2.189x)	3.660 (2.038x)

Table 8. Average execution time (milliseconds) of each scene. This experiment ablates the StopThePop [27] Tile-Based Culling method with warp-level load balancing against our AccuTile algorithm by breaking out the per-scene execution times, which were averaged in the Overall column of Table 7. The **fastest** times are highlighted. Our AccuTile algorithm outperforms Tile-Based Culling on all scenes.

Method	Mip-NeRF 360									Tanks & Temples		Deep Blending	
	bicycle	bonsai	counter	flowers	garden	kitchen	room	stump	treehill	train	truck	drjohnson	playroom
Baseline	14.034	4.977	7.025	7.914	6.103	11.025	8.562	5.776	7.088	6.993	4.872	7.253	5.322
Tile-Based Culling [27]	6.609	2.674	3.261	3.474	3.888	6.994	4.741	2.997	3.058	4.417	2.860	4.131	3.554
AccuTile (Ours)	5.880	2.401	2.989	2.933	3.478	6.433	4.490	2.578	2.726	3.965	2.729	3.671	3.300

A.4. StopThePop Tile-Based Culling Ablation

We ablate our AccuTile algorithm against the StopThePop [27] Tile-Based Culling method in Tables 7 and 8. Tile-Based Culling computes a precise Gaussian-to-Tile mapping in two steps: (1) Similar to our SnugBox method, a tight, opacity-aware bounding box is computed per Gaussian; however, due to the use of thresholds in their code, not all bounding boxes are tight. (2) Each tile touching the bounding box is iteratively examined to determine if it should be included in the final Gaussian-to-Tile mapping; warp-level load balancing is used to accelerate this process.

For this ablation, we update the 3D-GS rasterizer with the Tile-Based Culling code to isolate its runtime speed-up. All warp-level load balancing code is included to ensure that we compare against the most optimized version of the method. As noted in the StopThePop codebase, a padded alpha threshold is required to accurately compute bounding boxes, which, by extension, prevents undercounting Gaussian-to-Tile mappings. No padded alpha threshold is provided, so we perform this ablation without it. To ensure a fair comparison, we train three models for each scene and measure execution times with the baseline 3D-GS, Tile-Based Culling, and AccuTile renderers on each one.

Table 7 shows that our AccuTile method significantly outperforms Tile-Based Culling on Preprocess and Duplicate with Keys. Since Tile-Based Culling iterates over all candidate tiles while AccuTile does not, it requires more computation and induces a markedly higher runtime cost even with warp-level load-balancing. Surprisingly, Tile-Based Culling slightly outperforms AccuTile in the downstream functions Radix Sort, Identify Tile Ranges, and Render. However, we observe that this is caused by the aforementioned under-counting of Gaussian-to-Tile mappings;

this marginal improvement disappears when padded alpha thresholds are introduced, further slowing down Preprocess and Duplicate with Keys. Additionally, as reported by Table 8, our AccuTile method consistently outperforms Tile-Based Culling across all scenes.

A.5. Per-Scene Metrics

PSNR, SSIM, LPIPS, FPS, and training times for each scene from the Mip-NeRF 360, Tanks&Temples, and Deep Blending datasets that was used in 3D-GS [14] are recorded in Tables 9, 10, 11, 12, and Table 13, respectively. The operation in each row is applied **cumulatively** to all of the following rows.

Table 9. PSNR $\uparrow$ on each scene after cumulatively applying each function.

Method	Mip-NeRF 360									Tanks & Temples		Deep Blending	
	bicycle	bonsai	counter	flowers	garden	kitchen	room	stump	treehill	train	truck	drjohnson	playroom
Baseline	25.10	32.42	29.14	21.41	27.31	31.49	31.66	26.78	22.62	22.01	25.40	28.18	30.00
+SnugBox	25.12	32.36	29.09	21.45	27.31	31.61	31.70	26.78	22.54	21.97	25.41	28.27	30.09
+AccuTile	25.13	32.42	29.13	21.43	27.33	31.65	31.68	26.80	22.58	22.00	25.45	28.23	30.00
+Soft Pruning	25.09	31.91	28.74	21.35	27.16	30.83	31.32	26.88	22.57	21.74	25.34	28.44	30.14
+Hard Pruning	24.78	31.29	28.28	21.21	26.70	29.91	30.99	26.79	22.51	21.71	25.20	28.50	30.14

Table 10. SSIM $\uparrow$ on each scene after cumulatively applying each function.

Method	Mip-NeRF 360									Tanks & Temples		Deep Blending	
	bicycle	bonsai	counter	flowers	garden	kitchen	room	stump	treehill	train	truck	drjohnson	playroom
Baseline	0.747	0.948	0.916	0.589	0.857	0.933	0.927	0.770	0.636	0.815	0.883	0.880	0.891
+SnugBox	0.749	0.948	0.916	0.591	0.857	0.933	0.928	0.771	0.636	0.815	0.883	0.880	0.892
+AccuTile	0.749	0.948	0.916	0.590	0.857	0.933	0.927	0.771	0.637	0.816	0.883	0.879	0.891
+Soft Pruning	0.741	0.941	0.904	0.582	0.848	0.921	0.920	0.776	0.630	0.803	0.878	0.884	0.893
+Hard Pruning	0.704	0.927	0.878	0.561	0.815	0.894	0.905	0.765	0.590	0.773	0.868	0.882	0.892

Table 11. LPIPS $\downarrow$ on each scene after cumulatively applying each function.

Method	Mip-NeRF 360									Tanks & Temples		Deep Blending	
	bicycle	bonsai	counter	flowers	garden	kitchen	room	stump	treehill	train	truck	drjohnson	playroom
Baseline	0.244	0.183	0.185	0.359	0.122	0.118	0.200	0.242	0.346	0.208	0.147	0.291	0.284
+SnugBox	0.241	0.183	0.185	0.358	0.122	0.117	0.199	0.241	0.345	0.208	0.147	0.291	0.284
+AccuTile	0.242	0.183	0.185	0.359	0.122	0.117	0.199	0.241	0.344	0.207	0.147	0.292	0.284
+Soft Pruning	0.271	0.197	0.212	0.379	0.147	0.141	0.222	0.258	0.390	0.237	0.165	0.297	0.295
+Hard Pruning	0.333	0.231	0.260	0.419	0.213	0.198	0.260	0.288	0.463	0.291	0.191	0.313	0.308

Table 12. FPS $\uparrow$ on each scene after cumulatively applying each function. Speed-ups $\uparrow$ are recorded in (parentheses).

Method	Mip-NeRF 360									Tanks & Temples		Deep Blending	
	bicycle	bonsai	counter	flowers	garden	kitchen	room	stump	treehill	train	truck	drjohnson	playroom
Baseline	71	201	142	126	164	91	117	172	140	141	200	138	185
+SnugBox	154 (2.15 $\times$)	358 (1.78 $\times$)	276 (1.95 $\times$)	301 (2.39 $\times$)	267 (1.62 $\times$)	146 (1.60 $\times$)	197 (1.68 $\times$)	335 (1.95 $\times$)	301 (2.15 $\times$)	228 (1.61 $\times$)	320 (1.60 $\times$)	247 (1.79 $\times$)	282 (1.53 $\times$)
+AccuTile	168 (2.35 $\times$)	413 (2.05 $\times$)	330 (2.33 $\times$)	332 (2.64 $\times$)	285 (1.73 $\times$)	155 (1.70 $\times$)	221 (1.89 $\times$)	378 (2.20 $\times$)	315 (2.25 $\times$)	248 (1.75 $\times$)	343 (1.71 $\times$)	272 (1.97 $\times$)	294 (1.59 $\times$)
+Soft Pruning	241 (3.37 $\times$)	601 (2.99 $\times$)	505 (3.56 $\times$)	497 (3.95 $\times$)	419 (2.55 $\times$)	255 (2.80 $\times$)	425 (3.63 $\times$)	612 (3.56 $\times$)	549 (3.92 $\times$)	379 (2.68 $\times$)	518 (2.59 $\times$)	423 (3.06 $\times$)	477 (2.58 $\times$)
+Hard Pruning	662 (9.25 $\times$)	978 (4.87 $\times$)	842 (5.94 $\times$)	1122 (8.91 $\times$)	825 (5.02 $\times$)	640 (7.03 $\times$)	809 (6.90 $\times$)	1277 (7.42 $\times$)	942 (6.73 $\times$)	724 (5.12 $\times$)	1392 (6.95 $\times$)	957 (6.93 $\times$)	1149 (6.21 $\times$)

Table 13. Training time $\downarrow$ in minutes on each scene after cumulatively applying each function. Speed-ups $\uparrow$ are recorded in (parentheses).

Method	Mip-NeRF 360									Tanks & Temples		Deep Blending	
	bicycle	bonsai	counter	flowers	garden	kitchen	room	stump	treehill	train	truck	drjohnson	playroom
Baseline	31.9	20.4	24.1	24.1	32.3	27.8	23.7	24.1	24.2	11.1	13.4	24.8	19.5
+SnugBox	28.2 (1.13 $\times$)	19.2 (1.07 $\times$)	21.8 (1.11 $\times$)	22.7 (1.06 $\times$)	29.9 (1.08 $\times$)	25.8 (1.08 $\times$)	21.4 (1.11 $\times$)	22.9 (1.05 $\times$)	22.4 (1.08 $\times$)	9.8 (1.13 $\times$)	12.3 (1.09 $\times$)	21.7 (1.14 $\times$)	17.8 (1.09 $\times$)
+AccuTile	27.8 (1.15 $\times$)	19.0 (1.08 $\times$)	21.3 (1.13 $\times$)	22.6 (1.07 $\times$)	29.4 (1.10 $\times$)	25.5 (1.09 $\times$)	21.1 (1.12 $\times$)	22.7 (1.06 $\times$)	22.3 (1.08 $\times$)	9.7 (1.14 $\times$)	12.2 (1.09 $\times$)	21.5 (1.15 $\times$)	17.7 (1.10 $\times$)
+Soft Pruning	23.1 (1.38 $\times$)	17.0 (1.20 $\times$)	18.6 (1.30 $\times$)	19.5 (1.23 $\times$)	23.2 (1.39 $\times$)	20.3 (1.37 $\times$)	18.3 (1.30 $\times$)	19.3 (1.25 $\times$)	18.9 (1.27 $\times$)	8.3 (1.33 $\times$)	9.7 (1.38 $\times$)	17.3 (1.43 $\times$)	14.2 (1.37 $\times$)
+Hard Pruning	19.7 (1.62 $\times$)	16.0 (1.28 $\times$)	17.7 (1.36 $\times$)	17.5 (1.38 $\times$)	20.3 (1.59 $\times$)	18.7 (1.49 $\times$)	16.9 (1.40 $\times$)	17.1 (1.41 $\times$)	16.9 (1.43 $\times$)	7.2 (1.55 $\times$)	8.3 (1.61 $\times$)	15.3 (1.62 $\times$)	12.8 (1.52 $\times$)